



KUBESPECTRA

OPEN SOURCE INNOVATION – SECURE BY DESIGN

Kyverno Deep Dive: Policies debuggen, testen und überwachen

Marie Padberg

20.11.2025

Why KubeSpectra



We build, implement and develop Kubernetes-based container platforms.



Containerization



DevOps



Migration

We enable the quick, secure and flexible use of applications in the cloud.

Agenda

1. Kurze Kyverno Wiederholung
2. Implementierung einer Kyverno Validation Policy
3. Debuggen und Testen dieser Policy
4. Monitoring der implementierten Policy



Kyverno Recap - Policy Engine

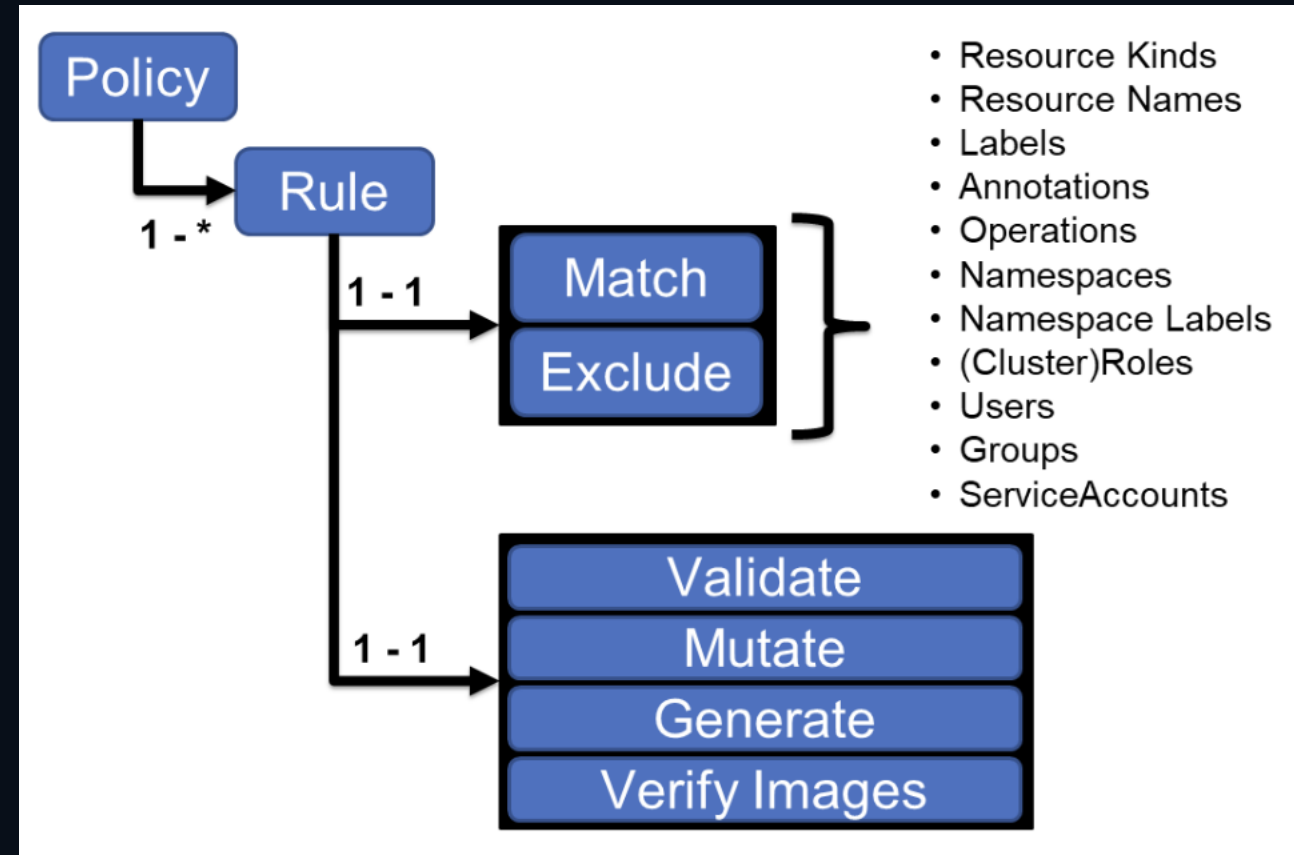
- Policy Engine für Kubernetes, die Richtlinien als Kubernetes Ressourcen managt
- Nutzt dynamische Kubernetes Admission Controller
- Kyverno umfasst 4 Arten von Richtlinien
 - Validierungen (validate)
 - Veränderungen (mutate)
 - Neue Kubernetes-Ressourcen basierend auf einer Richtlinie erstellen (generate)
 - Entfernen von Kubernetes Ressourcen (cleanup)
- Policy Reports bzgl. der Überprüfungen
- Anwendung auf sämtliche Kubernetes Objekte



Kyverno Recap - Policy Engine



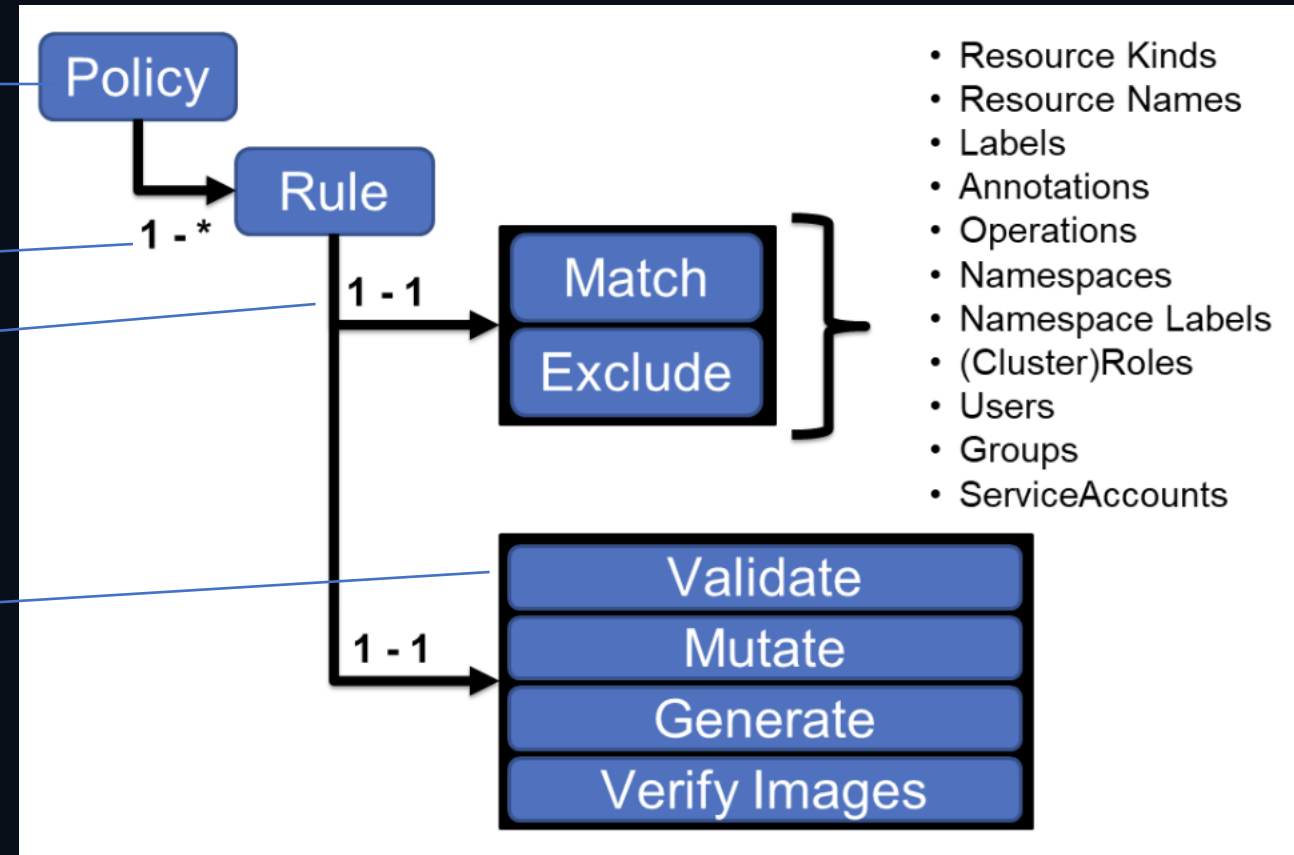
- Eine Kyverno Policy ist eine Sammlung von Regeln
- Jede Regel enthält eine match-Klausel und ggf. auch eine exclude-Klausel
- Clusterweit oder namespaceweit
- 2 Modi:
 - **Audit:** Verstöße gegen die Richtlinie werden vermerkt, sind aber ansonsten zulässig
 - **Enforce:** Verstöße gegen die Richtlinien führen zur Ablehnung



Kyverno Recap - Beispiel Validation Policy



```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
name: validate-image-tag
spec:
  rules:
  - name: deny-latest-tag
    match:
      any:
      - resources:
          kinds:
          - Pod
    validate:
      failureAction: Audit
      message: "Using the mutable image tag
'latest' is not allowed."
      pattern:
        spec:
          =(containers):
          - image: "!*:latest"
```



Es wird eine neue Policy benötigt...



- Anforderung: Stelle sicher, dass Namespaces über ein projectId-Label verfügen, das mit der erwarteten Projekt-ID aus einem vordefinierten Projekt-zu-Namespace-Mapping übereinstimmt, mit Ausnahme des default-Namespaces.

```
metadata:  
  name: team-x  
  labels:  
    projectId: pid12345
```

- Voraussetzung: Wir kennen das Mapping zwischen ProjectId und Namespace
- Umsetzung:
 - Policy muss das Mapping zwischen ProjectId und Namespace kennen
 - Namespaces ohne oder fehlerhafter ProjectId sollen abgelehnt werden
 - default-Namespace benötigt eine Ausnahme



Was ist JMESPath?

- JMESPath (ausgesprochen „James path“)
- Eine deklarative Abfragesprache für JSON-Daten
- Ermöglicht gezielte Extraktion & Filterung von Datenstrukturen

Warum ist das in Kyverno wichtig?

- Unverzichtbar in preconditions, deny-rules oder foreach-loops
- Strukturüberprüfung durch Pattern ist nicht immer ausreichend



Typische Einsatzbereiche

- Zugriff auf Labels, Annotations, Container-Images etc.
- Ermöglicht komplexe Bedingungen: `request.object.metadata.labels.app == 'frontend'`
- Iterationen & Filter: `spec.[initContainers, containers][].image`
- Message-Feld, falls eine Policy eine Ressource blockiert

Syntax-Kurzüberblick

- Immer `{{ }}` für JMESPath-Ausdrücke
- Nur `"{{...}}"`, wenn das Feld der alleinige Ausdruck ist, z.B. Label - `appns: "{{request.namespace}}"`
- Ausdruck + Text → Quotes optional, z.B. `message: "Namespace is {{request.namespace}}"`
- `.` = Sub-Expression → Zugriff auf verschachtelte Felder



Was bedeutet {{ request.namespace }}?

1. Erstellung Pod: `kubectl run nginx --image=nginx`
2. Kubernetes API-Server verarbeitet den Request
 - a) Validiert die YAML-Datei
 - b) Fügt systemseitige Standardwerte hinzu (z.B. Timestamps)
 - c) Verpackt die Anfrage/Request in ein AdmissionReview Objekt und sendet dieses an dynamische Admission Webhooks weiter
3. Das AdmissionReview-Objekt enthält u.A.
 - a) Die ursprüngliche Ressource
 - b) Informationen über die Aktion (CREATE, DELETE,...)
 - c) Namespace
 - d) Wer den Request ausgelöst hat
 - e) ...



- Auszug aus einem AdmissionReview

```
{
  "kind": "AdmissionReview",
  "apiVersion": "admission.k8s.io/v1",
  "request": {
    "name": "nginx",
    "namespace": "psa-test",
    "operation": "CREATE",
    "userInfo": {
      "username": "kubernetes-admin",
    },
    "object": {
      "kind": "Pod",
      "apiVersion": "v1",
      "metadata": {
        "name": "nginx",
        "labels": {...}
      },
    },
  },
}
```

JMESPath – Custom Filters



- Diese Filter erweitern die Standardfunktionen von JMESPath und helfen, typische Kubernetes-Anwendungsfälle zu lösen
- `contains()`-Filter
 - Gibt *true* zurück, wenn das angegebene *\$subject* die angegebene *\$search*-Zeichenkette enthält.
 - Beispiel: `contains('foobar', 'foo')` -> Ergebnis: *true*
 - Nützlich, um zu prüfen, ob ein bestimmtes Element oder Teilstring vorhanden ist
- `pattern_match()`-Filter
 - Vergleicht einfachen, non-regex Ausdruck, indem ein Eingabemuster und die zu vergleichende Zeichenfolge/Zahl angegeben werden
 - Ausgabe ist ein Boolean
 - Nützlich, wenn einfachere Vergleiche durchgeführt werden sollen, typischerweise mit Zeichenfolgen oder Zahlen



- Nützlich, um das Verhalten von Policies zu validieren und zu testen
- *jp* ist ein Unterbefehl der Kyverno CLI
 - Ermöglicht das Testen von JMESPath-Ausdrücken
- Liste der verfügbaren benutzerdefinierten JMESPath-Filter von Kyverno: `kyverno jp function`
- Weitere Beispiele:
 - *jp* → Einstieg in den "JMESPath"-Modus
 - *query* → führt den JMESPath-Ausdruck aus
 - `<JMESPath-Ausdruck>` → Filter oder Zugriffspfad im JSON-Dokument

```
kubectl get pod nginx -o json | kyverno jp query 'spec.containers[0].image'
```

```
kyverno jp query -i admissionreview.json 'request.namespace'
```




Unsere Policy: validate-ns-label-projectId

- *Anforderung: Stelle sicher, dass Namespaces über ein projectId-Label verfügen, das mit der erwarteten Projekt-ID aus einem vordefinierten Projekt-zu-Namespace-Mapping übereinstimmt, mit Ausnahme des default-Namespaces.*
- Benötigte JMESPath Ausdrücke:

```
request.object.metadata.labels bzw. request.object.metadata.labels.projectId  
request.object.metadata.name
```



Kyverno Validation Policies – Variablen

- Machen Policies dynamisch und wiederverwendbar, z. B. in Templates, Patchstrategien oder Validierungen
- Vordefinierte Variablen
 - `system:serviceaccount:<namespace>:<user>`
- Policies können sich selbst referenzieren, z. B. mit `$(../..)`, um Werte aus der Policy-Struktur wiederzuverwenden (ähnlich einem relativen Pfad)
 - Beispiel: *livenessProbe.port* wird auf denselben Wert wie *readinessProbe.port* gesetzt mittels `$(../..../readinessProbe/tcpSocket/port)`
- Einbinden von ConfigMaps, API-Server-Daten, Services oder Image-Registries als Variable mittels *context*

Unsere Policy: validate-ns-label-projectId



- *Anforderung: Stelle sicher, dass Namespaces über ein projectId-Label verfügen, das mit der erwarteten Projekt-ID aus einem vordefinierten Projekt-zu-Namespace-Mapping übereinstimmt, mit Ausnahme des default-Namespace.*
- ConfigMap in der Policy verwenden:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: project-namespace-mapping
  namespace: kyverno
data:
  pid12345: team-x
  pid67890: team-y
```

```
context:
- name: projectMap
  configMap:
    name: project-namespace-mapping
    namespace: kyverno
```



Kyverno Validation Policies – Deny Rules

- Erweiterung von validate-Rules: neben Pattern-Checks gibt es auch **deny**-Blöcke mit Bedingungen
- Wenn eine deny-Bedingung wahr (true) wird, wird die Ressource blockiert
- Innerhalb eines deny-Blocks:
 - **all** vs. **any**
 - **key**: JSON-Pfad oder Variablenzugriff (z. B. request.object.spec...).
 - **operator**: Vergleiche wie Equals, NotEquals, LessThan, AnyNotIn, ...
 - **value**: Vergleichswert, auch Listentypen
 - **message**: Optionale Fehlermeldung – wird im Admission-Response angezeigt

Unterschied zu Pattern-Validation:

- **Pattern** (Overlay): prüft auf Vorhandensein/Werte → blockiert bei Nicht-Vorhandensein
- **Deny**: Bedingung erfüllt → Workload wird geblockt

Unsere Policy: validate-ns-label-projectId



- *Anforderung: Stelle sicher, dass Namespaces über ein projectId-Label verfügen, das mit der erwarteten Projekt-ID aus einem vordefinierten Projekt-zu-Namespace-Mapping übereinstimmt, mit Ausnahme des default-Namespaces.*
- Benötigte Conditions:

```
deny:
conditions:
  any:
    - key: "{{ request.object.metadata.labels.projectId | | " }}"
      operator: Equals
      value: ""
      message: "The label projectId is missing"
    - key: "{{
      pattern_match(      '{{projectMap.data.{{request.object.metadata.labels.projectId}}}',
      '{{request.object.metadata.name}}')
      }}"
      operator: Equals
      value: false
      message: "Namespace has the wrong ProjectId."
```



Kyverno Validation Policies – Preconditions

- Steuern wann eine Regel ausgeführt wird
- Werden nach match/exclude, aber vor dem eigentlichen Regelkörper geprüft
- Wenn die Vorbedingung als falsch evaluiert wird, wird der Regelkörper übersprungen
- Da Preconditions oft Felder überprüfen, die optional sind, kann es notwendig sein, eine JMESPath-Syntax für Nicht-Existenzprüfungen (| | ") zu verwenden
- Können durch Kontextdaten erweitert werden

Unsere Policy: validate-ns-label-projectId



- *Anforderung: Stelle sicher, dass Namespaces über ein projectId-Label verfügen, das mit der erwarteten Projekt-ID aus einem vordefinierten Projekt-zu-Namespace-Mapping übereinstimmt, mit Ausnahme des default-Namespaces.*
- Ausnahme für default-Namespace:

```
preconditions:  
  all:  
    - key: "{{ request.object.metadata.name }}"  
      operator: AnyNotIn  
      value: "default"
```


Kyverno Validation Policies – Auto-Gen Rules



- Pods sind zentrale Objekte in Kubernetes, werden aber selten direkt erstellt, stattdessen verwalten höhere Controller wie Deployments oder StatefulSets die Pods
- Kyverno vereinfacht Richtlinienmanagement, indem es automatisch Regeln für diese Controller aus einer einzigen Pod-Regel generiert
 - allerdings nur, wenn die Regel ausschließlich auf Pods zielt
- Wird durch Annotation gesteuert
 - *pod-policies.kyverno.io/autogen-controllers: Deployment, Job*
 - *pod-policies.kyverno.io/autogen-controllers: none*
- Kyverno generiert Regeln nur automatisch, wenn in *match* und *exclude* ausschließlich „Pod“ als Ressourcentyp angegeben ist

Testen

Testen – Manuell mit CLI



```
kyverno test /path/to/folderContainingTestYamls
```

- Testet einen bestimmten Satz von Ressourcen gegen eine oder mehrere Policies, um die gewünschten Ergebnisse, die im Voraus in einer separaten Testmanifestdatei angegeben wurden, mit den tatsächlichen Ergebnissen zu vergleichen
- *kyverno test* sucht nach der Datei *kyverno-test.yaml* und führt die darin enthaltenen Tests aus
- In den Tests kann zwischen 4 Optionen entschieden werden (pass, skip, fail, warn). Wenn das Ergebnis des Tests die gewünschte Option ausgibt, gilt dieser als bestanden

Testen – Aufbau Test



- Auszug aus *kyverno-test.yaml*

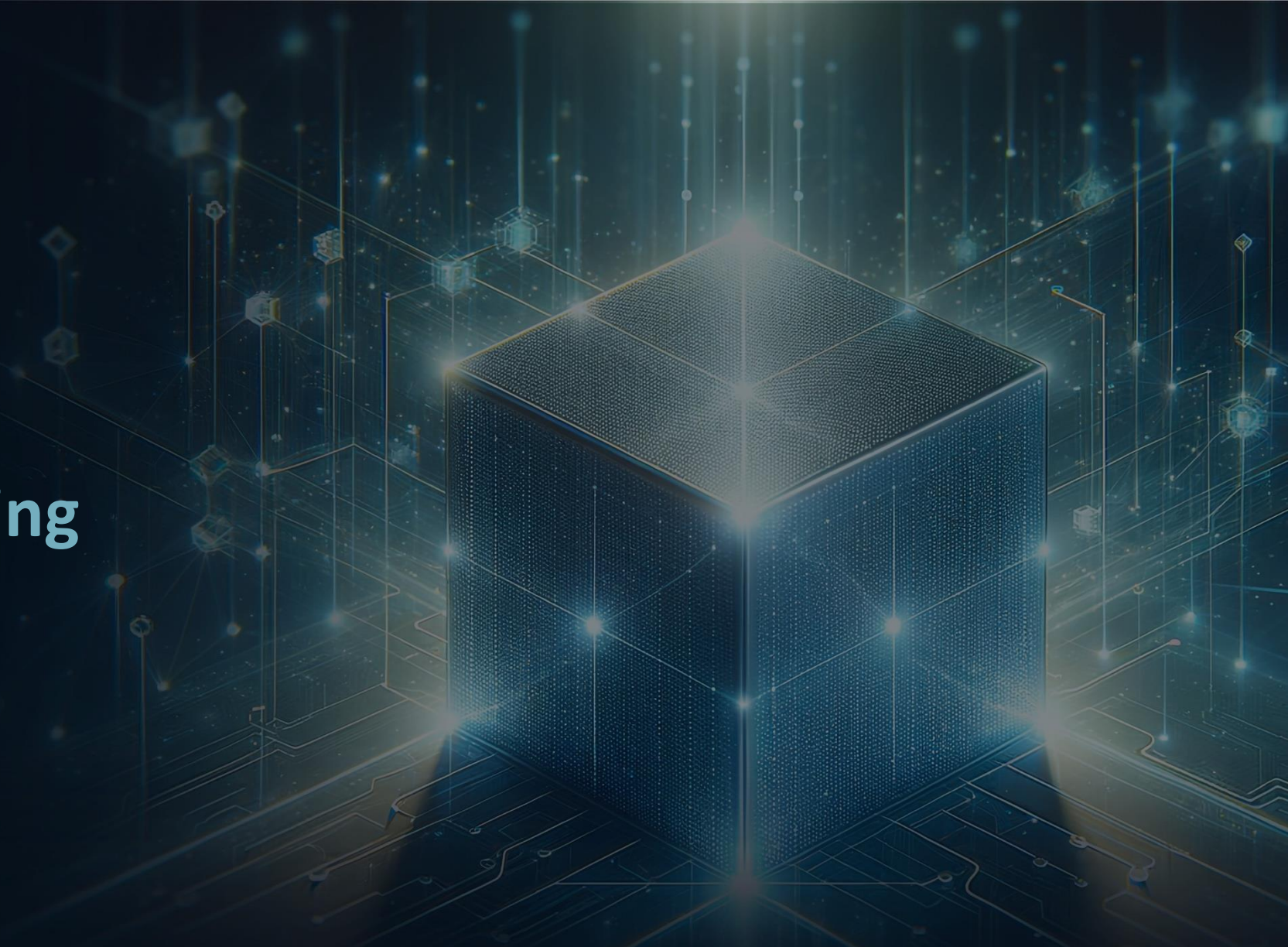
```
apiVersion: cli.kyverno.io/v1alpha1
kind: Test
metadata:
  name: validate-namespace
policies:
  - ../../policies/validate-namespace.yaml
resources:
  - namespace-success.yaml
  - namespace-fail.yaml
variables: variables.yaml
results:
  - policy: validate-namespace-projectid
    rule: validate-ns-label-projectid
    resources:
      - team-x
    kind: Namespace
    result: pass
```

Testen – Automatisierung mittels CI



- Zwei wesentliche Anwendungsfälle
 1. Entwicklungsteams sind verantwortlich für ihre Manifeste und möchten bei einem Pull-Request überprüfen, ob diese regelkonform sind, bevor ein Deployment stattfindet
 - Pipeline ist nur erfolgreich, wenn alle Tests bestanden sind
 - Tests, die hier nicht bestanden werden, werden auch im Kubernetes Cluster fehlschlagen
 2. Testen der Regeln
 - Verhalten der Policies überprüfen
 - Erwartetes Verhalten nach Änderungen sicherstellen
 - Neue Kyverno Versionen testen
- Umsetzung durch Nutzung einer Github Action: [kyverno/action-install-cli@v0.2.0](https://github.com/kyverno/kyverno-action)
- Das Git Repository muss die Policies, die zu testenden Ressourcen und die entsprechenden Tests enthalten

Reports & Monitoring



Reports - Beispiel Validation Policy



```
$ kubectl create deployment busybox --image=busybox:latest -n psa-test
```

```
$ kubectl get policyreport -n psa-test
```

NAME		KIND		NAME		PASS	FAIL	WARN	ERROR
SKIP	AGE								
43c9a204-37a4-4960-ba6b-c974ee29fb6d		Pod		deployment-check-example-7f854fdc76-					
zrdxz	0	1	0	0	0				99s

```
apiVersion: wgpolicyk8s.io/v1alpha2
kind: PolicyReport
results:
- message: 'validation error: Using the mutable image tag "latest" is not allowed.
  rule deny-latest-tag failed at path /spec/containers/0/image/'
  policy: validate-image-tag
  result: fail
  rule: deny-latest-tag
  scored: true
  source: kyverno
scope:
  apiVersion: v1
  kind: Pod
  name: deployment-check-example-7f854fdc76-zrdxz
  namespace: psa-test
summary:
  error: 0
  fail: 1
  pass: 0
  skip: 0
  warn: 0
```

Reports – Background Scans



- Die Anwendung von Policies auf vorhandene Ressourcen wird als Background Scan bezeichnet und ist standardmäßig aktiviert
- Der Background Scan erfolgt in regelmäßigen Abständen (standardmäßig stündlich).
- Bei aktivem Background Scan werden die Ergebnisse in einem Report aufgezeichnet, unabhängig davon, ob der Modus Enforce oder Audit ist
 - Es werden keine vorhandenen Ressourcen blockiert, auch nicht im Enforce-Modus

```
spec:  
  background: true  
  rules:  
  - name: deny-image-tag
```


Reports – Policy Reporter



- Policy Reports
 - Über alle Namespaces verteilt
 - Unübersichtlichkeit in Reports, da viele Ergebnisse in einem Bericht stehen
 - Aufwändig fehlgeschlagene Regeln zu finden
- Policy Reporter
 - Wurde entwickelt, um die Ergebnisse der validierenden Kyverno Policies besser sichtbar und beobachtbar zu machen
 - Der optionale Metrik-Endpunkt kann verwendet werden, um Verstöße in Monitoring-Tools wie Grafana zu beobachten
 - Bietet auch ein eigenständiges Dashboard, um einen grafischen Überblick über alle Ergebnisse und Informationen über Kyverno-Richtlinien zu erhalten

Ausblick und Fazit



- Verständnis von JMESpath und Kenntnisse über das Admissionreview-Objekt sind entscheidend
- CLI zum Testen und Experimentieren
- Policy Reporter für einen schnellen Überblick
- Viele Wege führen nach Rom: Policy kann auch anders geschrieben werden, z.B. ohne `pattern_match`, oder Precondition in den `deny`-Block ziehen, ...
- Es wurde nur eine Auswahl populärer Funktionen gezeigt – Kyverno bietet noch viele weitere
- Link zum Git Repo: <https://github.com/kubespectra/kyverno-workshop>

LinkedIn





KubeSpectra is a brand of infologistix GmbH.
All rights reserved.

infologistix GmbH
Gutenbergstraße 7
85748 Garching
Germany

www.kubespectra.io / www.infologistix.de
kubespectra@infologistix.de
T +49 89 81885979